



**UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO**

Herramientas de Programación Aplicada III (.Net)

Fecha del Lab:
14 – Septiembre -2026

II SEMESTRE 2026

Grupo: 1IL133

Instructor: Ing. Irina Fong

 Laboratorio: Introducción a Base de Datos con MySQL

Módulo:

 A. Título de la Experiencia

Desarrollo de una Aplicación CRUD en C# con Conexión a MySQL y Manejo Binario de Imágenes

 B. Temas

- ❖ Programación Orientada a Objetos (POO) aplicada en Windows Forms.
- ❖ Conexión y operaciones básicas (CRUD) con bases de datos MySQL.
- ❖ Uso de estructuras de datos (Dictionary<string, object>) para el paso de parámetros.
- ❖ Manipulación de flujos de memoria (MemoryStream) para la conversión y persistencia de imágenes.
- ❖ Interacción con el usuario mediante cuadros de diálogo (OpenFileDialog) y componentes visuales (DataGridView y PictureBox).

 C. Objetivos

1. **Implementar** una interfaz gráfica en C# Windows Forms que permita capturar datos textuales y gráficos de un producto.



2. **Programar** funciones de conversión de tipos de datos (como transformar objetos Image a arreglos de bytes mediante MemoryStream y RawFormat) para su almacenamiento en bases de datos relacionales.
3. **Establecer** la comunicación segura entre la aplicación de escritorio y una tabla en MySQL para automatizar la inserción y el refresco de registros en tiempo real en un DataGridView.

D. Metodología

1. **Práctica guiada basada en proyectos:** Se trabaja de forma incremental sobre un sistema real de inventario de productos, resolviendo problemas técnicos y de sintaxis directamente sobre el código en ejecución.
2. **Aprendizaje constructivista:** Los estudiantes experimentan el flujo completo de los datos (desde los controles TextBox y PictureBox, pasando por diccionarios y flujos de memoria, hasta la persistencia final en la base de datos).
3. **Depuración y análisis de errores:** Se fomenta la lectura y comprensión de mensajes del compilador (excepciones, tipos de datos y nulabilidad estricta) como parte fundamental del proceso de desarrollo de software.

E. Requisitos del Laboratorio

Instalar el **MySQL Installer** y **MySQL Workbench**.

URL: <https://dev.mysql.com/downloads/installer/>

Tener el motor de base de datos nativo junto con su interfaz gráfica oficial (Workbench) les da el entorno exacto que se usa a nivel empresarial.

Para asegurarte de que todo corra bien en sus máquinas, solo ten en cuenta lo siguiente:

- **El servicio corriendo:** Asegúrate de recordarles que el servicio de MySQL (MySQL80 o el que se haya instalado) debe estar activo en los servicios de Windows, o que el *MySQL Notifier* esté encendido para que la base de datos responda cuando lancen su aplicación de C#.
- **La cadena de conexión:** Como todos están usando la misma versión oficial de MySQL, la estructura de la *connection string* en su código (apuntando a

localhost, con su usuario root y la contraseña que hayan configurado) les funcionará a todos exactamente igual.

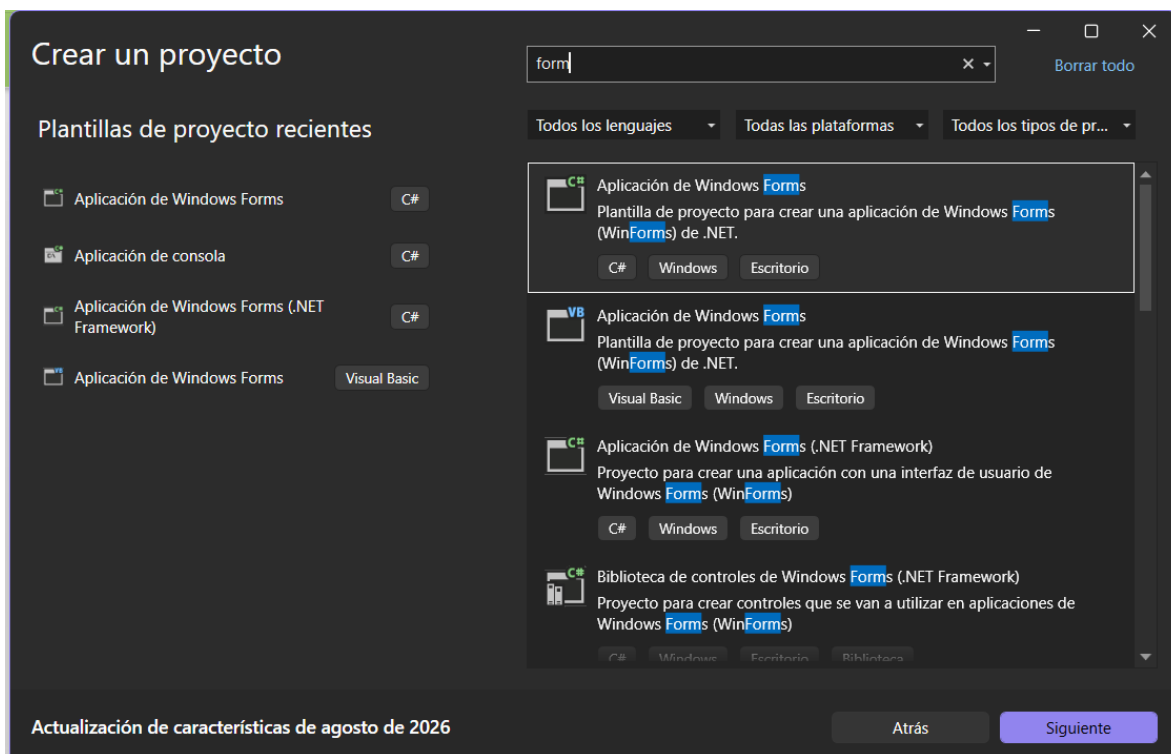
services.msc

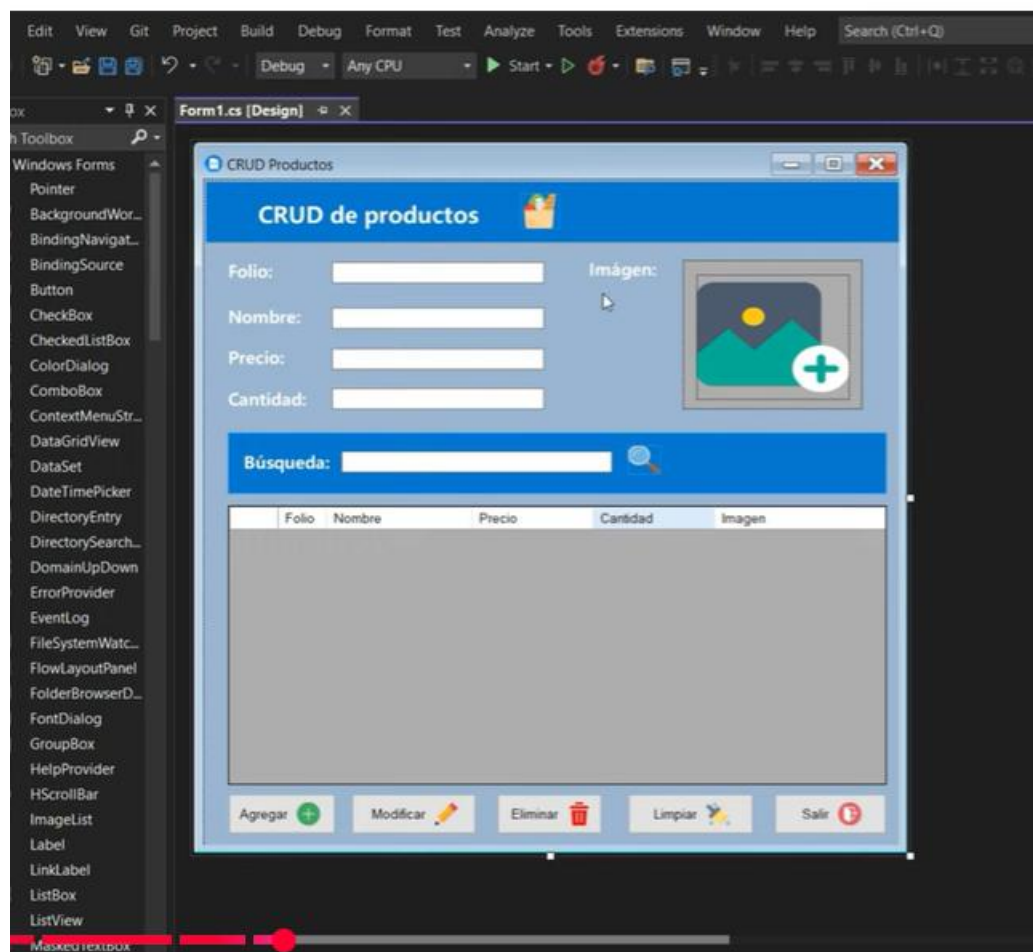
ese comando abre la consola de administración de los **Servicios de Windows**, que es el lugar perfecto para verificar si el motor de base de datos está encendido y funcionando.

No necesitan instalar ningún Connector/.NET por separado en Windows como una aplicación o instalador externo.

Esa parte la realiza **NuGet**, la cual permite descargar las librerías necesarias y guardarlas dentro de la carpeta del proyecto (en la carpeta packages o dentro de dependencias), de modo que cuando descarguen o clonen el código fuente y lo abran en Visual Studio, el proyecto ya viene con el conector integrado y listo para compilar.

Aplicación de Windos Forms



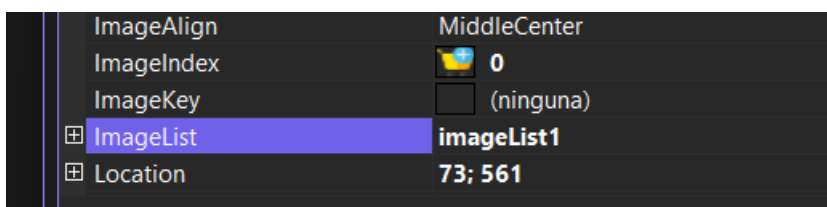
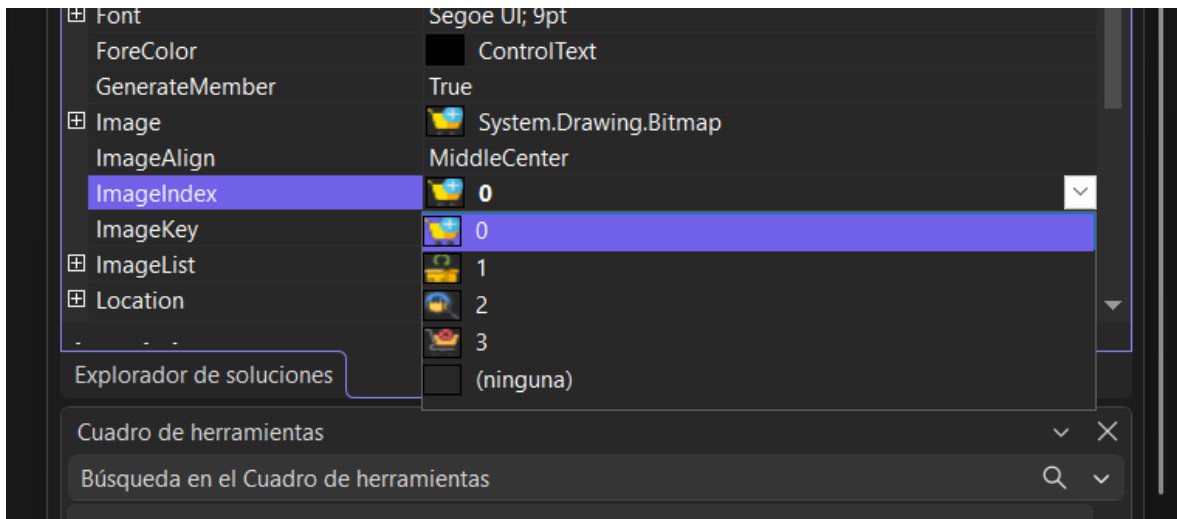
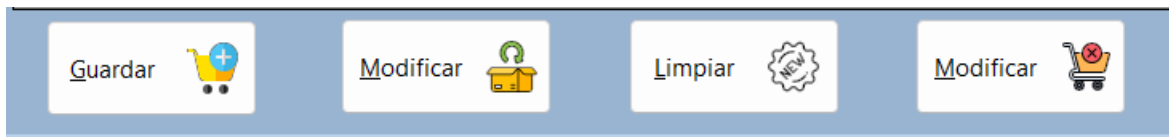


1. Agregar los campos de textos
2. Agregar un GridView
3. Agregar un PictureBox para la imagen de los productos
4. Agregar un ImageList

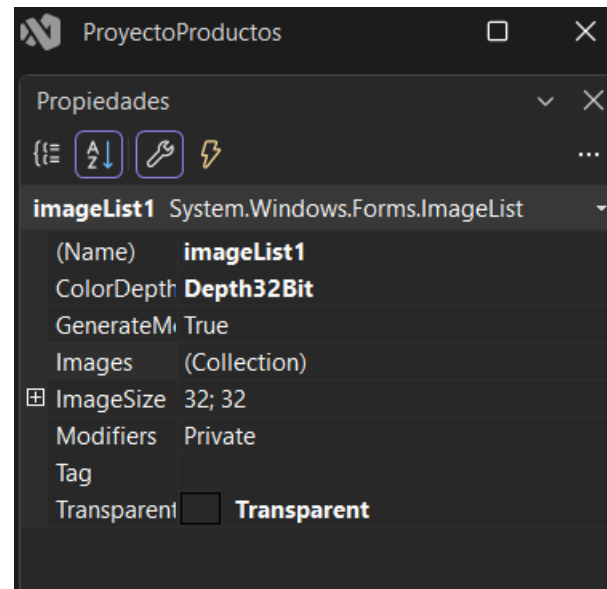
Vamos a vincular las los botones (button) con las imágenes del control ImageList:

Control ImageList

Vinculados a los botone guardar, modificar, limpiar.



Relacionar ImageList con la imagen, através del ImageIndex



1.Agregar ImageList:Paso 1.

Arrastra un componente **ImageList** desde el Cuadro de Herramientas a tu formulario. Aparecerá en el área de componentes no visuales (debajo del diseñador).

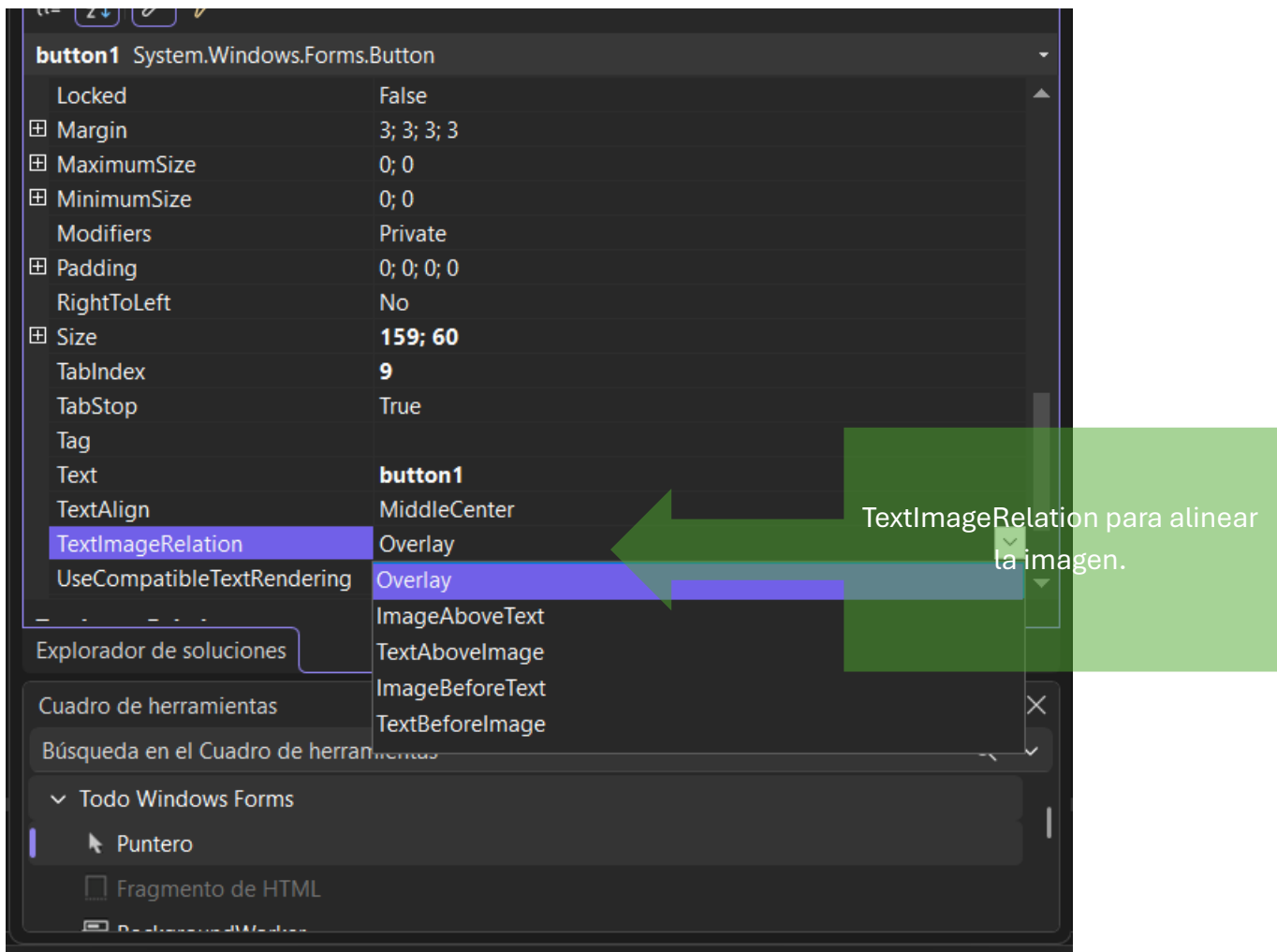
Verificación: Verás el ícono de un pequeño rollo de película con el nombre imageList1.

2.Configurar tamaño y cargar imagen:Paso 2.

1. Haz clic en la flecha inteligente (triángulo) del imageList1.
2. Selecciona "**Elegir imágenes**".
3. En la ventana que se abre, haz clic en "**Agregar**" y selecciona tu archivo del lápiz (asegúrate de que la imagen original sea pequeña de antemano, ej. 16x16 o 24x24 píxeles).
4. Cierra la ventana.
5. **Crucial:** En la ventana de Propiedades del imageList1, busca la propiedad **ImageSize** y cámbiala al tamaño deseado (ej. 16, 16 o 24, 24). Esto forzará a la imagen a adoptar ese tamaño.

3.Asociar con el botón:Paso 3.

1. Selecciona tu botón en el formulario.
2. En la ventana de Propiedades, busca la propiedad **ImageList** y selecciona `imageList1` en el menú desplegable.
3. Busca la propiedad **ImageIndex** y selecciona el índice de la imagen del lápiz que cargaste (usualmente será el 0 si solo tienes una).
4. Opcional: Ajusta la propiedad `TextImageRelation` a `ImageBeforeText` para que el ícono quede a la izquierda y el texto a la derecha.





El control PictureBox

El control **PictureBox** en Windows Forms está diseñado específica y únicamente **para mostrar imágenes**.

Es el contenedor estándar que usas en tu interfaz para que el usuario pueda ver una foto de perfil, un logo, un icono, etc.

Cómo se usa normalmente:

1. **Arrastras** un PictureBox desde el Cuadro de Herramientas a tu formulario.
2. **Cargas la imagen:**
 - **En tiempo de diseño (estática):** Haces clic en la flecha inteligente del control -> "Elegir imagen" y seleccionas un archivo de tu PC.
 - **En tiempo de ejecución (dinámica):** Mediante código, le dices qué archivo cargar o qué imagen mostrar.

Para regular cómo se muestra la imagen dentro de un PictureBox, la propiedad clave que debes ajustar es **SizeMode**. Esta propiedad determina cómo se ajusta la imagen al tamaño del control que has dibujado en el formulario.

Aquí tienes las opciones más comunes y para qué sirve cada una:

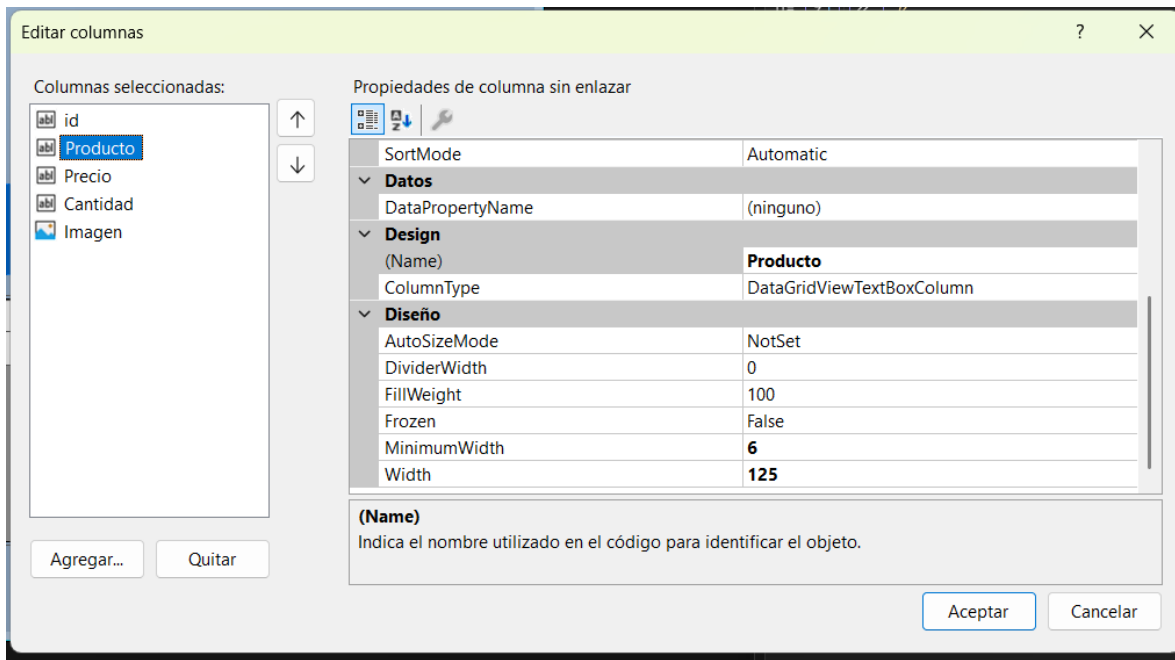
Las opciones de SizeMode:

Opción de SizeMode	Descripción visual y uso
Normal (Predeterminada)	La imagen se coloca en la esquina superior izquierda. Si la imagen es más grande que el PictureBox, se corta . Si es más pequeña, queda espacio vacío alrededor.
StretchImage	La imagen se estira o encoge para llenar completamente el área del PictureBox. Advertencia: Esto puede deformar la imagen si las proporciones originales no coinciden con el tamaño del control.
AutoSize	El PictureBox cambia de tamaño automáticamente para ajustarse a las dimensiones exactas de la imagen . Es útil si quieres mostrar la imagen en su tamaño real sin que se corte, pero el formulario puede desordenarse.
CenterImage	La imagen se centra dentro del PictureBox. Si la imagen es más grande, se corta por los bordes. Si es más pequeña, queda centrada con espacio vacío alrededor.
Zoom (La más recomendada)	La imagen se ajusta para que quepa completamente dentro del PictureBox manteniendo sus proporciones originales (sin deformarla). Quedará espacio vacío (letras o franjas) si las formas no coinciden, pero la imagen nunca se distorsionará.

Control DataGridView

La mayoría de las columnas son DataGridViewTextBoxColumn

La imagen es un DataGridViewImageColm. Ajustes en la propiedad Width en 125.



Agregar un control rejilla al formulario.

Agregar columna ? X

☐ Columna de enlace de datos

Columnas en DataSource

☒ Columna sin enlazar

Nombre:

Tipo:

Texto del encabezado:

☒ Visible ☐ Solo lectura ☐ Inmovilizar

Editar columnas ? X

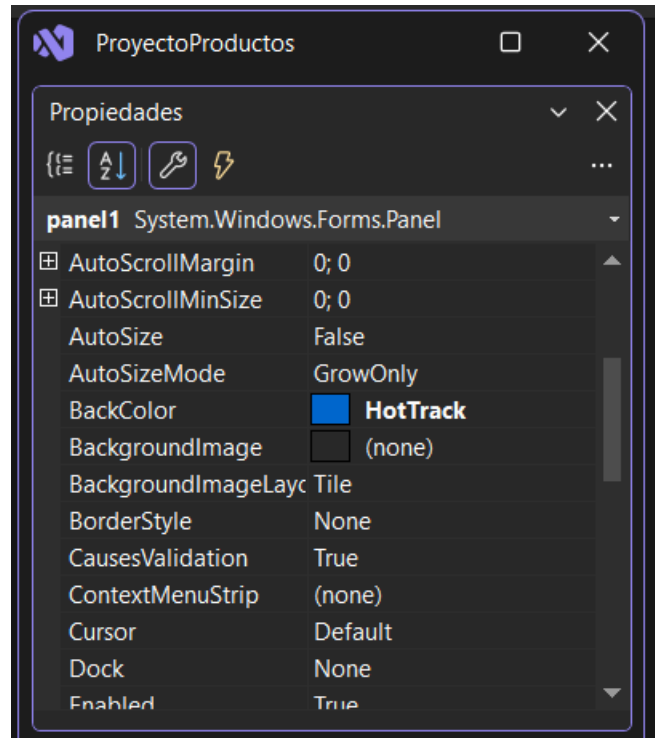
Columnas seleccionadas:

Propiedades de columna sin enlazar

ToolTipText	
Visible	True
Comportamiento	
ContextMenuStrip	(none)
ReadOnly	False
Resizable	True
SortMode	Automatic
Datos	
DataPropertyName	(ninguno)
Design	
(Name)	dataGridViewTextBoxColumn1
ColumnType	DataGridViewImageColumn
Diseño	
AutoSizeMode	NotSet
DividerWidth	0
FillWeight	100
Frozen	False
MinimumWidth	6
Width	125

ColumnType
Tipo DataGridViewColumn.

TxtBusqueda.Text, está dentro de un panel que se le cambió la propiedad



Datos en la Base de Datos

Servidor: MySQL 3306 > Base de datos: productosdb > Tabla: productos

Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones

Estructura de tabla Vista de relaciones

#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Comentarios	Extra	Acción
☐ 1	id	int			No	Ninguna		AUTO_INCREMENT	Cambiar Eliminar Más
☐ 2	nombre	varchar(100)	utf8mb4_unicode_ci		No	Ninguna			Cambiar Eliminar Más
☐ 3	precio	decimal(10,2)			No	Ninguna			Cambiar Eliminar Más
☐ 4	cantidad	int			No	Ninguna			Cambiar Eliminar Más
☐ 5	imagen	longblob			No	Ninguna			Cambiar Eliminar Más

✓ Mostrando filas 0 - 5 (total de 6, La consulta tardó 0,0039 segundos.)

`SELECT * FROM `productos``

☐ Perfilando [[Editar en línea](#)] [[Editar](#)] [[Explicar SQL](#)] [[Crear código PHP](#)] [[Actualizar](#)]

☐ Mostrar todo | Número de filas: 25 | Filtrar filas: Ordenar según

Opciones extra

				id	nombre	precio	cantidad	imagen
☐	Editar	Copiar	Borrar	1	Teclado	20.00	2	
☐	Editar	Copiar	Borrar	2	Mouse	15.70	2	
☐	Editar	Copiar	Borrar	3	Portátil	700.70	2	[BLOB - 21.1 KB]
☐	Editar	Copiar	Borrar	4	Celular	100.00	2	[BLOB - 21.1 KB]
☐	Editar	Copiar	Borrar	5	teclado y mouse	40.00	5	[BLOB - 21.1 KB]
☐	Editar	Copiar	Borrar	6	teclado y mouse ve2	40.00	5	[BLOB - 11.0 KB]

```

DROP TABLE IF EXISTS `productos`;
CREATE TABLE IF NOT EXISTS `productos` (
  `id` int NOT NULL AUTO_INCREMENT,
  `nombre` varchar(100) CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci NOT NULL,
  `precio` decimal(10,2) NOT NULL,
  `cantidad` int NOT NULL,
  `imagen` longblob NOT NULL,
  PRIMARY KEY (`id`)
);

```

Crear la Clase Producto.cs

Prop Enter: genera la propiedad.

```

1  using System;
2  using System.Collections.Generic;
3
4  using System.Text;
5
6  namespace ProyectoProductos
7  {
8      7 referencias
9      public class Producto
10     {
11         2 referencias
12         public int Id { get; set; }
13         2 referencias
14         public string Nombre { get; set; }
15         2 referencias
16         public decimal Precio { get; set; }
17         2 referencias
18         public int Cantidad { get; set; }
19
20         4 referencias
21         public byte[] Imagen { get; set; }
22     }
23 }
    
```

Tipo de Dato Imagen

Imagen	Para guardar una imagen proveniente de un campo de tipo LONGBLOB de MySQL, el tipo de datos correcto que debes usar en tu clase de C# es un arreglo de bytes (byte[]), no un string. Un campo LONGBLOB almacena datos binarios puros (los ceros y unos de la imagen), por lo que en .NET se mapea directamente a byte[]. Ejemplo: int[] colocar los corchetes después del tipo de dato.
--------	--

Administrar paquetes NuGet

Paquete MySql.Data

using MySql.Data.MySqlClient;

- **Administrar paquetes NuGet** es exactamente lo que necesitas hacer cuando te falta el **conector de MySQL** (el paquete **MySql.Data**). Lo usas para descargar e instalar librerías externas que tu proyecto necesita para funcionar.
- **ALT + Enter** (o hacer clic en el foquito que aparece al lado de una palabra en rojo) es un atajo de Visual Studio para **resolver errores de código rápidos**, como por ejemplo agregar automáticamente el using **MySql.Data.MySqlClient**; arriba de tu archivo cuando te sale subrayado en rojo porque no encuentra la clase.

Así que para solucionar un problema de falta de conexión o librería:

1. Primero usas **Administrar paquetes NuGet** para instalar el paquete de MySQL si aún no lo tienes en el proyecto.

Atajos
ALT + Enter

1. Luego, si al escribir tu código te marca error porque no reconoce alguna clase o namespace, usas **ALT + Enter** para que Visual Studio agregue el using correspondiente de forma automática.

Crear la Conexión con la Base de Datos MySQL

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Text;
using MySql.Data.MySqlClient;

namespace ProyectoProductos
{
    public class Conexion
    {
        private static string cadenaConexion = "Server=localhost; Database=productosdb;Uid=root;Pwd=demo";

        public static MySqlConnection ObtenerConexion()
        {
            try
            {
                //Crear un tipo de dato de MySqlConnection
                MySqlConnection conexion = new MySqlConnection(cadenaConexion);
                conexion.Open();
                return(conexion);
            }
            catch (MySqlException ex) {

                Console.WriteLine("Error al conectar: " + ex.Message);
                return null;

            } //fin del catch

        } // fin dle método estático MySqlConnection
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Text;
using MySql.Data.MySqlClient;

namespace ProyectoProductos
{
    public class Conexion
    {

        private static string cadenaConexion = "Server=localhost;
        Database=productosdb;Uid=root;Pwd=demo";

        public static MySqlConnection ObtenerConexion()
        {

            try
            {
                //Crear un tipo de dato de MySqlConnection
                MySqlConnection conexion = new MySqlConnection(cadenaConexion);
                conexion.Open();
                return(conexion);
            }
            catch (MySqlException ex) {

                Console.WriteLine("Error al conectar: " + ex.Message);
                return null;

            }
        }

    }

}

// fin dle método estático MySqlConnection
```

Método Estático GetProductos

```
public static List<Producto> GetProductos(string filtro)
{
    List<Producto> listaProductos = new List<Producto>();

    string query = "SELECT id, nombre, precio, cantidad, imagen FROM productos";

    // Si viene un filtro, modificamos el query de forma segura
    // (Nota: idealmente con parámetros, pero adaptado al ejemplo visual que tienes)
    if (!string.IsNullOrEmpty(filtro))
    {
        query += " WHERE id LIKE @filtro OR nombre LIKE @filtro " +
            " OR precio LIKE @filtro OR cantidad LIKE @filtro";
    }

    using (MySqlConnection conn = ObtenerConexion())
    {
        if (conn == null) return listaProductos;

        using (MySqlCommand cmd = new MySqlCommand(query, conn))
        {
            // Si hay filtro, agregamos el parámetro para evitar inyección SQL
            if (!string.IsNullOrEmpty(filtro))
            {
                cmd.Parameters.AddWithValue("@filtro", "%" + filtro + "%");
            }

            // Ejecutamos el lector de datos
            using (MySqlDataReader mReader = cmd.ExecuteReader())
            {
                // Recorremos el lector fila por fila mientras haya registros
                while (mReader.Read())
                {
                    Producto prod = new Producto();

                    // Mapeamos los campos de la base de datos a las propiedades de tu clase Producto
                    // (Ajusta los nombres de las columnas o índices según tu base de datos)

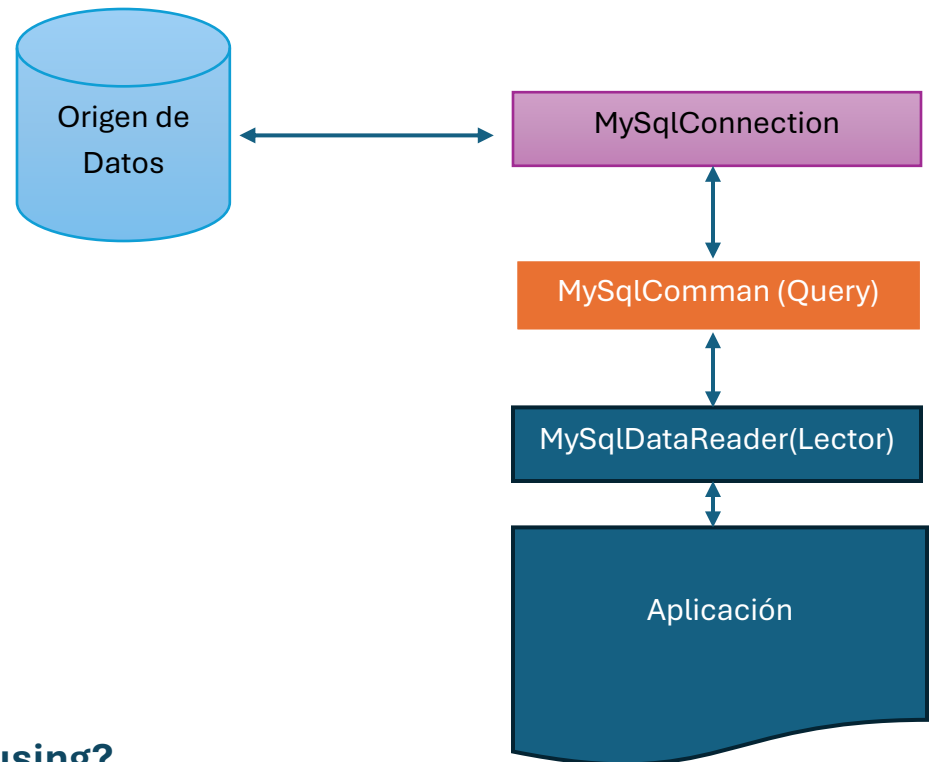
                    prod.Id = Convert.ToInt32(mReader["id"]);
                    prod.Nombre = mReader["nombre"].ToString();
                    prod.Precio = Convert.ToDecimal(mReader["precio"]);
                    prod.Cantidad = Convert.ToInt32(mReader["cantidad"]);
                    //prod.Imagen = (byte[])mReader.GetValue(4);

                    prod.Imagen = mReader["imagen"] != DBNull.Value ? (byte[])mReader["imagen"] : null;

                    // Agregamos el objeto listo a la lista genérica
                    listaProductos.Add(prod);
                }
                mReader.Close();
            } //MySqlDataReader
        } // MySqlCommand
    }

    return listaProductos;
}

//fin del método listaProductos
```



¿Por qué usamos using?

El using se utiliza para la **gestión automática de recursos del sistema (limpieza de memoria)**.

Gran parte de los objetos gráficos de Windows Forms (Bitmap, MemoryStream, fuentes, conexiones de bases de datos) consumen recursos pesados que escapan del recolector de basura normal de C# porque manejan memoria nativa del sistema operativo (lo que se conoce como recursos no administrados o *unmanaged resources*).

- **Sin using:** Si creas un Bitmap o un MemoryStream y olvidas cerrar o liberar sus espacios, esos bloques de memoria se quedan atrapados en la RAM causando fugas de memoria (*memory leaks*), ralentizando la aplicación o bloqueando archivos.
- **Con using:** Le dices a C# que **tan pronto como termine el bloque de código encerrado entre llaves {}**, el objeto se destruye automáticamente y libera de inmediato toda la memoria RAM y los recursos del sistema operativo que estaba utilizando.

```
public static List<Producto> GetProductos(string filtro)
{
    List<Producto> listaProductos = new List<Producto>();
    string query = "SELECT id, nombre, precio, cantidad, imagen FROM productos";

    // Si viene un filtro, modificamos el query de forma segura
    // (Nota: idealmente con parámetros, pero adaptado al ejemplo visual que tienes)
    if (!string.IsNullOrEmpty(filtro))
    {
        query += " WHERE id LIKE @filtro OR nombre LIKE @filtro " +
            " OR precio LIKE @filtro OR cantidad LIKE @filtro";
    }

    using (MySqlConnection conn = ObtenerConexion())
    {
        if (conn == null) return listaProductos;
        using (MySqlCommand cmd = new MySqlCommand(query, conn))
        {
            // Si hay filtro, agregamos el parámetro para evitar inyección SQL
            if (!string.IsNullOrEmpty(filtro))
            {
                cmd.Parameters.AddWithValue("@filtro", "%" + filtro + "%");
            }

            // Ejecutamos el lector de datos
            using (MySqlDataReader mReader = cmd.ExecuteReader())
            {
                // Recorremos el lector fila por fila mientras haya registros
                while (mReader.Read())
                {
                    Producto prod = new Producto();

                    // Mapeamos los campos de la base de datos a las propiedades de tu clase Producto
                    // (Ajusta los nombres de las columnas o índices según tu base de datos)

                    prod.Id = Convert.ToInt32(mReader["id"]);
                    prod.Nombre = mReader["nombre"].ToString();
                    prod.Precio = Convert.ToDecimal(mReader["precio"]);
                    prod.Cantidad = Convert.ToInt32(mReader["cantidad"]);
                    //prod.Imagen = (byte[])mReader.GetValue(4);

                    prod.Imagen = mReader["imagen"] != DBNull.Value ? (byte[])mReader["imagen"] : null;
                    // Agregamos el objeto listo a la lista genérica
                    listaProductos.Add(prod);
                }
                mReader.Close();
            }
        }
    }
}
```

Conversión a Enteros (Tipos de Datos Enteros)

Int32 (de Convert.ToInt32):

Maneja enteros de 32 bits, lo que corresponde exactamente al tipo int que definiste en tu tabla de MySQL para el id y la cantidad. Su rango va desde aproximadamente -2,147,483,648 hasta 2,147,483,647

Int16 (de Convert.ToInt16):

Maneja enteros de 16 bits (equivalente a un short en C#), cuyo valor máximo es de apenas **32,767**.

```
// Recorremos el lector fila por fila mientras haya registros
while (mReader.Read())
{
    Producto prod = new Producto();

    // Mapeamos los campos de la base de datos a las propiedades de tu clase
    // (Ajusta los nombres de las columnas o índices según tu base de datos)

    prod.Id = Convert.ToInt32(mReader["id"]);
    prod.Nombre = mReader["nombre"].ToString();
    prod.Precio = Convert.ToDecimal(mReader["precio"]);
    prod.Cantidad = Convert.ToInt32(mReader["cantidad"]);
    //prod.Imagen = (byte[])mReader.GetValue(4);

    prod.Imagen = mReader["imagen"] != DBNull.Value ? (byte[])mReader["imagen"] : null;

    // Agregamos el objeto listo a la lista genérica
    listaProductos.Add(prod);
}
```

Datos
mReader["producto"]
son los datos de
la Tabla Produto.

Los datos que se agregan en comillas dobles dentro del mReader deben coincidir con los nombres de columnas.

Conversión Explícita

Se hace una conversión explícita para el tipo de dato imagen a (byte[])

Una **conversión explícita** (también llamada *casting*) es cuando obligas al programa a transformar un dato de un tipo a otro de manera manual, porque el lenguaje de programación no lo puede hacer por sí solo de forma automática debido a que hay riesgo de perder información o causar un error.

```
double salarioDecimal = 1500.75;  
int salarioEntero = (int)salarioDecimal; // Conversión explícita con el operador (int)
```

Por qué se llama "explícita":

- **Porque tú se lo pides explícitamente al código** colocando el tipo de destino entre paréntesis (tipo) justo antes del valor.

En resumen: la conversión explícita es cuando **tú asumes la responsabilidad** de transformar el dato y le fuerzas la mano al sistema usando paréntesis.

Programación en el Formulario

Acciones:

Inicializar Colecciones

Cargar Datos al Grid

Creamos una lista tipo Producto, que utilizaremos para traer los registros de la tabla productos.

El propósito del Diccionario es para agregar registros a la base de datos (Insertar).

```
using System.Drawing.Imaging;

namespace ProyectoProductos
{
    public partial class Form1 : Form
    {
        //Cre Producto para Guardar los datos.
        //private Producto myNuevoProducto = new Producto();

        private List<Producto> listaProductos;
        private Dictionary<string, object> myProducto = new Dictionary<string, object>
    };
    public Form1()
    {
        InitializeComponent();
        listaProductos = new List<Producto>();
    }

    private void Form1_Load(object sender, EventArgs e)
    {
        cargarProductos();
    }
}
```

Es necesario llenar la rejilla para ello diríjase a la ventana de propiedades, haga clic en el botón Eventos y seleccione el evento Load que se producirá en el momento de cargar el formulario. Si hace doble clic sobre este evento se añadirá un manejador que responderá a este evento ejecutando el método Form1_Load.

Atajos Control +

Presionando Control + podemos de escribir el código cargarProductos en método Load, nos permite crear el método.

```
private void cargarProductos(string filtro = "")
{
    dgvProductos.Rows.Clear();
    dgvProductos.Refresh();
    listaProductos = Conexion.GetProductos(filtro);

    foreach (var prod in listaProductos)
    {
        Image img = null;

        if (prod.Imagen != null && prod.Imagen.Length > 0)
        {
            using (MemoryStream ms = new MemoryStream(prod.Imagen))
            {
                using (Bitmap bmp = new Bitmap(ms))
                {
                    img = new Bitmap(bmp); // Esto clona la imagen y evita que falle
                }
            }
        }

        dgvProductos.Rows.Add(prod.Id, prod.Nombre, prod.Precio, prod.Cantidad, img);
    } //fin del foreach listaProductos
} //fin de cargarProductos
```

¿Qué es MemoryStream?

Un **MemoryStream** es una clase de C# (System.IO.MemoryStream) que sirve para **crear y administrar un búfer o flujo temporal de datos directamente en la memoria RAM** de la computadora, en lugar de guardarlos en un archivo físico en el disco duro.

Puntos clave:

- **Almacenamiento Temporal en RAM:** Actúa como un contenedor en la memoria principal para escribir y leer bytes de manera rápida y eficiente sin dejar rastros de archivos temporales en el disco.
- **Puente para Conversión de Imágenes:** En el sistema de productos, funciona como el mecanismo de traducción indispensable para convertir una imagen visual (extraída de un PictureBox) en un arreglo de bytes (byte[]), y viceversa.
- **Compatibilidad con la Base de Datos:** Permite empaquetar archivos gráficos usando su formato original (RawFormat) para que puedan viajar de manera



segura como datos binarios y ser almacenados directamente en campos tipo BLOB de MySQL.

¿Por qué usamos Bitmap?

Un **Bitmap** es una clase de C# (System.Drawing.Bitmap) que representa una imagen compuesta por una cuadrícula de píxeles (un mapa de bits). Es el objeto especializado que te permite cargar, manipular, modificar y guardar imágenes píxel por píxel en la memoria.

En este método de carga, el uso de Bitmap (junto con el MemoryStream) cumple un rol fundamental de conversión y desvinculación para poder mostrar las imágenes que vienen de la base de datos dentro del DataGridView.

El desglose paso a paso de por qué se usa exactamente así en nuestro código es el siguiente:

1. Lectura de bytes (MemoryStream): La base de datos te devuelve la imagen convertida en un arreglo de bytes (prod.Imagen). Como el DataGridView no entiende de arreglos de bytes crudos, pasas esos bytes a un MemoryStream para que C# pueda leerlos como un flujo de datos en la memoria RAM.
2. Construcción del gráfico (new Bitmap(ms)): El objeto Bitmap toma esos bytes del MemoryStream y los reconstruye visualmente en una imagen gráfica que Windows Forms sí puede renderizar en pantalla.
3. El truco de la clonación (img = new Bitmap bmp): Esta línea es clave. Cuando un MemoryStream se cierra (lo cual pasa inmediatamente al salir del bloque using (MemoryStream ms = ...)), el flujo de memoria se destruye. Si pasaras el bmp directo al grid, al cerrarse el ms la imagen fallaría o daría error de recursos. Al hacer un segundo new Bitmap bmp, clonas la imagen creando una copia independiente que vive libremente en la memoria RAM, permitiendo cerrar el MemoryStream de forma segura sin perder la visualización en la tabla.

Inicializar Colecciones

```
using System.Drawing.Imaging;

namespace ProyectoProductos
{
    public partial class Form1 : Form
    {
        //Cre Producto para Guardar los datos.
        //private Producto myNuevoProducto = new Producto();

        private List<Producto> listaProductos;
        private Dictionary<string, object> myProducto = new Dictionary<string, object>();
        public Form1()
        {
            InitializeComponent();
            listaProductos = new List<Producto>();
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            cargarProductos();
        }
    }
}
```

Cargar Productos

```
private void cargarProductos(string filtro = "")
{
    dgvProductos.Rows.Clear();
    dgvProductos.Refresh();
    listaProductos = Conexion.GetProductos(filtro);

    //Recorrido de la Lista de Productos
    foreach (var prod in listaProductos)
    {
        Image img = null;

        if (prod.Imagen != null && prod.Imagen.Length > 0)
        {
            using (MemoryStream ms = new MemoryStream(prod.Imagen))
            {
                using (Bitmap bmp = new Bitmap(ms))
                {
                    img = new Bitmap(bmp); // Esto clona la imagen y evita que falle
                }
            }
        }

        dgvProductos.Rows.Add(prod.Id, prod.Nombre, prod.Precio, prod.Cantidad, img);
    }
}
```

La gran diferencia entre las dos formas de llenar un DataGridView en C# Windows Forms: **añadir filas de manera manual** versus **enlazarlo a una fuente de datos**.

1. Método manual (dgvProductos.Rows.Add)

Es el que acabas de escribir.

- **Cómo funciona:** Tú creas las filas una por una pasándole los valores exactos a mano (prod.Id, prod.Nombre, etc.). Incluso puedes meter objetos complejos como imágenes (img) directamente en una celda de tipo DataGridViewImageColumn.

- **Cuando se usa:** Cuando los datos vienen sueltos, de consultas muy personalizadas, o cuando mezclas tipos de datos que no vienen de una sola tabla o lista directa.
- **Desventaja:** Si el usuario edita una celda en pantalla, eso **no** se actualiza automáticamente en tu objeto original; tendrías que programar el código para leer la celda y actualizar tu base de datos manualmente.

```
dgvProductos.Rows.Add(prod.Id, prod.Nombre, prod.Precio, prod.Cantidad, img);
```

2. Método enlazado (dgvProductos.DataSource)

Es el que usamos en el laboratorio anterior, la vez pasada (por ejemplo, pasándole un List<Producto> o un BindingSource).

- **Cómo funciona:** Le tiras la lista completa de objetos al DataSource y el DataGridView se encarga solito de leer las propiedades (Id, Nombre, Precio, etc.) y armar las columnas y filas de golpe.
- **Cuando se usa:** Es la forma más profesional y limpia cuando trabajas con Bases de Datos o listas en memoria con Programación Orientada a Objetos.
- **Ventaja:** Funciona en doble vía (si usas un BindingList); lo que cambias en la tabla se refleja en tu objeto y viceversa. Con las imágenes a veces es un poquito más mañoso configurarlo.

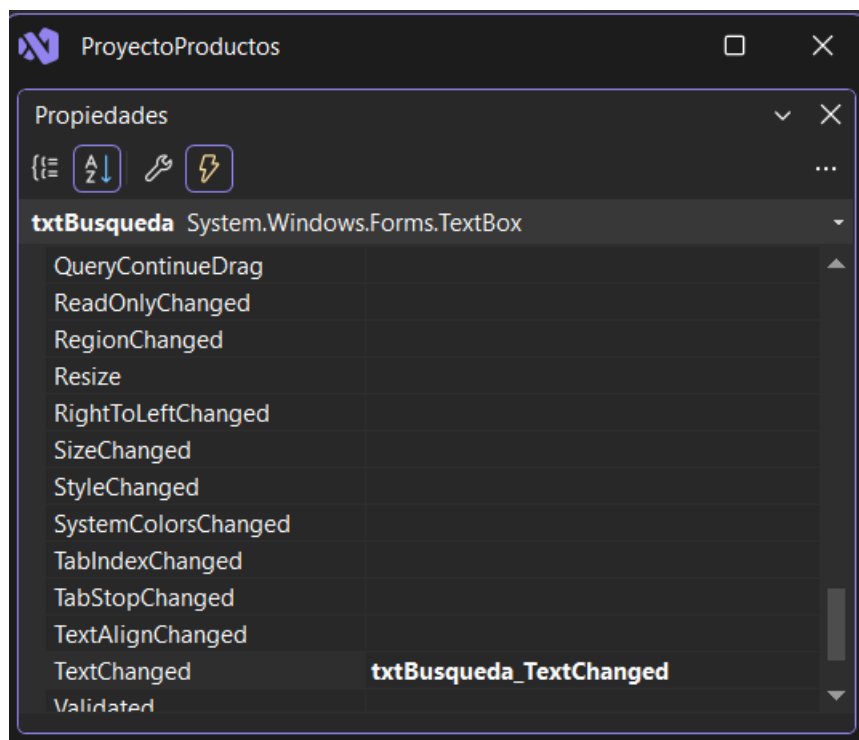
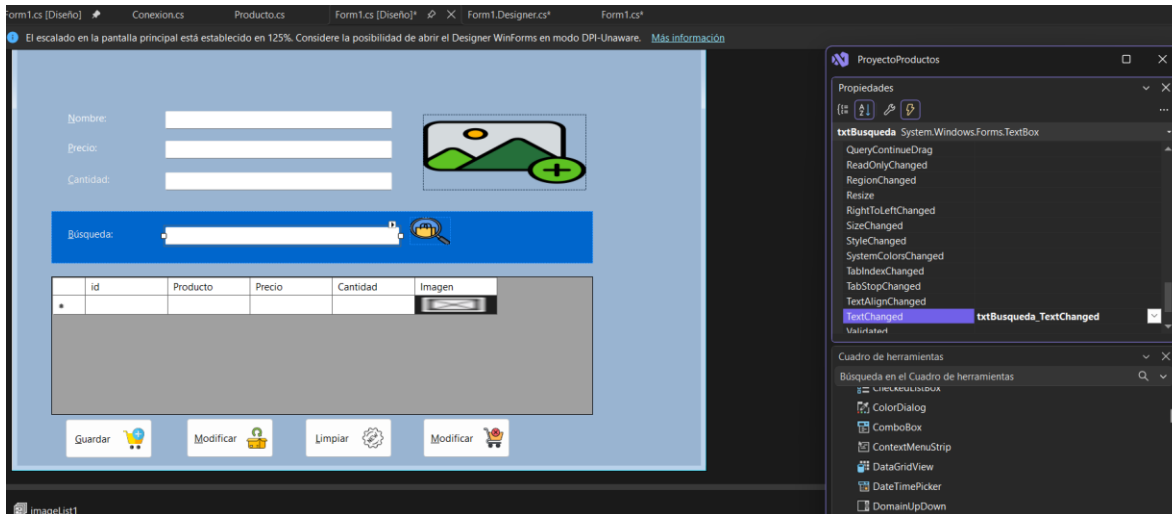
```
Persona colaborador1 = new Persona();
colaborador1.Id = int.Parse(txtID.Text);
colaborador1.Nombres = txtNombres.Text;
colaborador1.Apellidos = txtApellidos.Text;
colaborador1.Correo = txtCorreo.Text;
colaborador1.Salario = salario1;
colaborador1.FechaNacimiento = dtpFechaNacimiento.Value;

listaPersonas.Add(colaborador1);
dgvdatos.DataSource = null; // Limpiar el DataSource antes de asignar la nueva lista
dgvdatos.DataSource = listaPersonas;

txtApellidos.Text = string.Empty;
txtCorreo.Text = "";
```

Hasta este punto hemos cargado los registros que están en la base de datos a la rejilla (grid).

Buscar algún dato en el grid



Doble clic sobre el Evento TextChanged para agregar el código necesario para buscar o cargar los datos de la base de datos al grid pero con filtro.

```
private void txtBusqueda_TextChanged(object sender, EventArgs e)
{
    cargarProductos(txtBusqueda.Text.Trim());
}
```

Cargar una Imagen del pictureBox1

pictureBox1

```

private void pictureBox1_Click(object sender, EventArgs e)
{
    using (OpenFileDialog openFileDialog = new OpenFileDialog())
    {
        openFileDialog.Title = "Seleccionar imagen del producto";
        openFileDialog.Filter = "Archivos de imagen|*.jpg;*.jpeg;*.png;*.bmp";

        if (openFileDialog.ShowDialog() == DialogResult.OK)
        {
            // Carga la imagen seleccionada en el PictureBox y ajusta su tamaño
            pictureBox1.Image = Image.FromFile(openFileDialog.FileName);
            pictureBox1.SizeMode = PictureBoxSizeMode.StretchImage;
        }
    }
}

```

OpenFileDialog

El **OpenFileDialog** es un componente nativo de Windows Forms (System.Windows.Forms.OpenFileDialog) que invoca la ventana exploradora de archivos estándar del sistema operativo. Su propósito en la arquitectura de la aplicación es permitir que el usuario navegue por su disco duro de forma gráfica y controlada para seleccionar un archivo específico.

Para incluirlo formalmente en la documentación de tu laboratorio, puedes estructurarlo con los siguientes puntos clave basados en el código:

- **Propósito general:** Actúa como un puente interactivo entre la interfaz de escritorio y el sistema de archivos local, garantizando que la aplicación obtenga una ruta de archivo válida sin requerir ingreso manual por texto.
- **openFileDialog.Title:** Propiedad que personaliza el encabezado o título de la ventana flotante del explorador para orientar al usuario sobre qué tipo de archivo debe elegir (por ejemplo, "Seleccionar imagen del producto").
- **openFileDialog.Filter:** Mecanismo de restricción que limita la visualización de archivos en el explorador según su extensión (por ejemplo, *.jpg;*.png;*.bmp), evitando que el usuario seleccione archivos incompatibles que puedan corromper o generar excepciones en el sistema.
- **ShowDialog() == DialogResult.OK:** Estructura de control condicional que verifica si el usuario seleccionó un archivo de manera exitosa y presionó el botón de aceptación, asegurando que el flujo continúe únicamente si hay una ruta válida disponible.
- **openFileDialog.FileName:** Propiedad que almacena la ruta absoluta en el disco del archivo seleccionado, la cual es consumida directamente por el método Image.FromFile() para cargar la representación visual en el PictureBox y ajustar su escala mediante PictureBoxSizeMode.StretchImage.



```
private void btnGuardar_Click(object sender, EventArgs e)
{
    if (!datosCorrectos())
    {
        return; // No vamos hacer nada - se detiene en este punto, puedes crear un punto de interrupción
    }

    CargarDatosProductos();

    if (Conexion.InsertSeguro("productos", myProducto)) ;
    {
        MessageBox.Show("Se ha guardado satisfactoriamente el registro");
        // Aquí refrescas el grid volviendo a consultar la base de datos
        cargarProductos();
    } //fin del if InsertSeguro
}
```

```
private void CargarDatosProductos()
{
    myProducto["Cantidad"] = int.Parse(txtCantidad.Text.Trim());
    myProducto["Precio"] = decimal.Parse(txtPrecio.Text.Trim());
    myProducto["Nombre"] = txtNombre.Text.Trim();
    myProducto["Imagen"] = ImageToByteArray(pictureBox1.Image);
}

private byte[] ImageToByteArray(Image? image)
{
    if (image == null)
        return null;

    using (MemoryStream mMemoryStream = new MemoryStream())
    {
        // Guardamos la imagen usando su formato original (RawFormat)
        image.Save(mMemoryStream, image.RawFormat);
        return mMemoryStream.ToArray();
    }
}
```

```
private bool datosCorrectos()  
{  
    if (txtNombre.Text.Trim().Equals(""))  
    {  
        MessageBox.Show("Ingrese el Nombre del Producto");  
        return false;  
    }  
  
    if (txtPrecio.Text.Trim().Equals(""))  
    {  
        MessageBox.Show("Ingrese el Precio");  
        return false;  
    }  
  
    if (txtCantidad.Text.Trim().Equals(""))  
    {  
        MessageBox.Show("Ingrese la Cantidad");  
        return false;  
    }  
  
    if (!decimal.TryParse(txtPrecio.Text.Trim(), out decimal precio))  
    {  
        MessageBox.Show("Ingrese un Precio correcto");  
        return false;  
    }  
  
    if (!int.TryParse(txtCantidad.Text.Trim(), out int cantidad))  
    {  
        MessageBox.Show("Ingrese una cantidad correcto");  
        return false;  
    }  
  
    return true;  
}
```



decimal.TryParse

decimal.TryParse es un método seguro en C# que se utiliza para **convertir una cadena de texto (string) en un valor numérico de tipo decimal**, evitando que la aplicación se detenga o lance una excepción si el usuario ingresa un texto inválido, letras o deja el campo vacío.

Para documentarlo técnicamente en tus materiales de clase, se compone de los siguientes puntos clave:

- **Manejo seguro de errores:** A diferencia de métodos como decimal.Parse(), que provocan un error crítico (crash) si el texto no se puede convertir, TryParse evalúa la conversión de forma controlada.
- **Valor de retorno booleano:** Devuelve true si el texto pudo convertirse con éxito a número, o false si la conversión falló.
- **Parámetro de salida (out):** Utiliza la palabra clave out para entregar el número ya convertido en una nueva variable lista para usarse.

Out

```
if (!decimal.TryParse(txtPrecio.Text.Trim(), out decimal precio))
{
    MessageBox.Show("Ingrese un Precio correcto");
    return false;
}

if (!int.TryParse(txtCantidad.Text.Trim(), out int cantidad))
{
    MessageBox.Show("Ingrese una cantidad correcto");
    return false;
}
```

Sin embargo, en el caso específico de `int.TryParse`:



Normalmente, una función en programación solo puede devolver un único resultado a través de la instrucción `return`. En el caso de `int.TryParse`:

1. Su **resultado principal** (el `return` de la función) es un booleano (`true` o `false`) que indica si la conversión tuvo éxito o fracasó.
2. Como también necesitamos obtener el número ya convertido (si la conversión fue exitosa), se utiliza el modificador **out** como un canal secundario de salida.

Puntos clave:

- **Paso por referencia y asignación obligatoria:** out le indica a C# que la variable cantidad se pasará para que el método la llene con el valor numérico procesado. El método está obligado a asignarle un valor antes de que termine su ejecución.



- **Declaración en línea (*Inline declaration*):** Al escribir `out int` cantidad, C# permite crear la variable en el mismo instante en que se llama al método, limitando su uso y facilitando una sintaxis limpia.
- **Disponibilidad posterior:** Si `TryParse` devuelve `true`, esa variable cantidad queda inmediatamente disponible para ser utilizada en el bloque de código siguiente (por ejemplo, para guardarla en tu objeto producto o enviarla a la base de datos) con la seguridad de que contiene un número válido.

Método que guarda los datos

```
public static bool InsertSeguro(string tbName, Dictionary<string, object> data)
{
    var columns = string.Join(", ", data.Keys);
    var placeholders = "@" + string.Join(", @", data.Keys);

    string sql = $"INSERT INTO {tbName} ({columns}) VALUES ({placeholders})";

    try
    {
        // Pedimos la conexión usando nuestra clase externa
        using (MySqlConnection conexion = ObtenerConexion())
        {
            if (conexion == null) return false;

            using (MySqlCommand stmt = new MySqlCommand(sql, conexion))
            {
                foreach (var kvp in data)
                {
                    stmt.Parameters.AddWithValue("@ " + kvp.Key, kvp.Value ??
DBNull.Value);
                }

                stmt.ExecuteNonQuery();
                return true;
            }
        }
    }
    catch (MySqlException ex)
    {
        Console.WriteLine("Error en INSERT: " + ex.Message);
        return false;
    }
}

} //fin del método Insertar Seguro
```

```
public static bool InsertSeguro(string tbName, Dictionary<string, object> data)
{
    var columns = string.Join(", ", data.Keys);
    var placeholders = "@" + string.Join(", @", data.Keys);

    string sql = $"INSERT INTO {tbName} ({columns}) VALUES ({placeholders})";

    try
    {
        // Pedimos la conexión usando nuestra clase externa
        using (MySQLConnection conexion = ObtenerConexion())
        {
            if (conexion == null) return false;

            using (MySQLCommand stmt = new MySQLCommand(sql, conexion))
            {
                foreach (var kvp in data)
                {
                    stmt.Parameters.AddWithValue("@ " + kvp.Key, kvp.Value ?? DBNull.Value);
                }

                stmt.ExecuteNonQuery();
                return true;
            }
        }
    }
    catch (MySQLException ex)
    {
        Console.WriteLine("Error en INSERT: " + ex.Message);
        return false;
    }
}
//fin del método Insertar Seguro
```

Punto de interrupción

Cómo verificar que sí la está recibiendo:

Colocar un punto de interrupción (breakpoint) es muy sencillo y es la herramienta principal para depurar:

1. Ve al código de tu formulario (Form1.cs).
2. Haz clic con el ratón exactamente en el margen gris izquierdo (a la par del número de la línea donde dice foreach (var prod in listaProductos)).
3. Aparecerá un círculo rojo y la línea se pondrá de color rojo.

Cuando ejecutes el programa (F5) y el formulario cargue, la ejecución se pausará justo ahí antes de que ocurra cualquier error, permitiéndote pasar el cursor sobre las variables o presionar Shift + F9 para inspeccionar su contenido en tiempo real.

Para comprobar que los datos llegaron correctamente al formulario sin adivinar, pon un punto de interrupción (breakpoint) justo en esta línea de tu formulario:

C#

```
foreach (var prod in listaProductos)
```

Cuando el programa se detenga ahí, selecciona la palabra listaProductos, presiona **Shift + F9** (Inspección rápida) y despliega los elementos. Verás que la lista ya trae todos los productos cargados desde la base de datos listos para pintarse en el DataGridView.

Haz clic directamente sobre el círculo rojo en el margen izquierdo donde lo pusiste, y el punto de interrupción desaparecerá de inmediato.

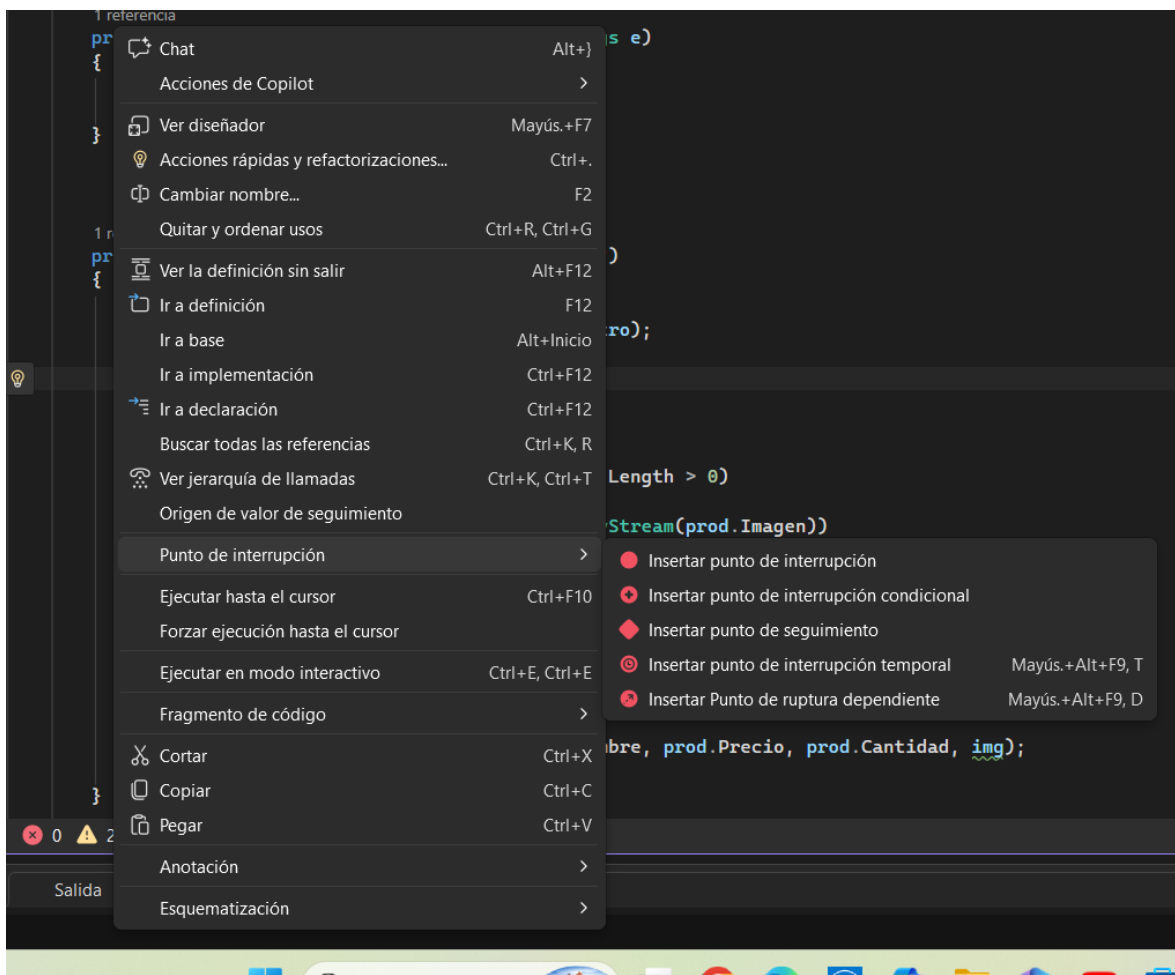
También puedes colocar el cursor del teclado en esa misma línea y presionar la tecla **F9** (o **Fn + F9** en tu laptop) para quitarlo.

Si en algún momento quieres borrar **todos** los puntos de interrupción de todo el proyecto de golpe, puedes presionar las teclas **Ctrl + Shift + F9**.

En la gran mayoría de las laptops, las teclas de función (F1 al F12) comparten espacio con otras funciones multimedia (como subir volumen, brillo, etc.). Para presionar **F5**, tienes dos caminos principales:

1. **Usa la tecla Fn:** Mantén presionada la tecla **Fn** (generalmente ubicada en la esquina inferior izquierda, cerca de la barra espaciadora o la tecla Ctrl) y, sin soltarla, presiona la tecla **F5** en la parte superior.
2. **Bloqueo de función (Fn Lock):** Si no quieres estar presionando Fn todo el tiempo, puedes buscar en tu teclado si hay una combinación como **Fn + Esc** (a veces tiene un pequeño icono de "Fn Lock" o un candado). Al presionarla una vez, las teclas superiores se invertirán y funcionarán directamente como F1 al F12 sin necesidad de usar la tecla Fn.

Clic derecho:



```
Form1.cs [Diseño]  Conexion.cs  Producto.cs  Form1.cs [Diseño]  Form1.Designer.cs  Form1.cs
ProyectoProductos
113     }
114
115     if (txtCantidad.Text.Trim().Equals(""))
116     {
117         MessageBox.Show("Ingrese la Cantidad");
118         return false;
119     }
120
121     if (!decimal.TryParse(txtPrecio.Text.Trim(), out decimal precio))
122     {
123         MessageBox.Show("Ingrese un Precio correcto");
124         return false;
125     }
126
127     if (!int.TryParse(txtCantidad.Text.Trim(), out int cantidad))
128     {
129         MessageBox.Show("Ingrese una cantidad correcto");
130         return false;
131     }
132
133     return true;
134 }
135 }
136 }
137 }
```



Acceso a Datos

Si se conecta a una base de datos de Microsoft SqlServer, puede mejorar el rendimiento general utilizando SqlDataAdapter junto con sus objetos asociados SqlCommand y SqlConnection. En el caso de la base de datos de Oracle, utilice OracleDataAdapter junto con sus objetos OracleCommand y OracleConnection.

Objeto lector de datos

Cuando una aplicación sólo necesite leer datos (no actualizarlos), no será necesario almacenarlos en un conjunto de datos, basta utilizar un objeto lector de datos en su lugar. Un objeto lector de datos obtiene los datos del origen de dato y los pasa directamente a la aplicación (un adaptador de datos utiliza un objeto lector de dato para llenar un conjunto de datos).

El objeto lector de datos proporcionado por .NET para SQL Server es SqlDataReader y para orígenes OLE-DB es OleDbDataReader.

Objeto orden

Después de establecer una conexión con un origen de datos, puede utilizar un objeto Command para ejecutar sentencias SQL y devolver resultados desde ese origen de datos. Para crear un objeto de éstos, invoque a su constructor. Si el objeto ya está creado, la instrucción SQL encapsula por él puede ser consultada o modificada a través de la propiedad CommandText.

Para el caso de Microsoft SQL Server, utilice SqlCommand y para Oracle, OracleCommand.

Adaptadores de datos

Un adaptador es un conjunto de objetos utilizados para intercambiar datos entre un origen de datos y un conjunto de datos (objeto DataSet). Esto significa que una aplicación leerá datos de una base de datos para un conjunto de datos y, a continuación, manipulará dichos datos. También en algunas ocasiones, volverá a escribir en la base de datos los datos modificados del conjunto de datos.

Si se conecta a una base de datos de Microsoft SQL Server, puede mejorar el rendimiento general utilizando SqlDataAdapter junto con sus objetos SqlCommand y SqlConnection.

Generalmente, cada adaptador de datos, intercambia datos entre una sola tabla de un origen de datos y un solo objeto DataTable (tabla de datos) del conjunto de datos.

Imágenes

El nombre del parámetro (image con minúscula) es solo una variable, así que puedes llamarlo como quieras (por ejemplo, img), pero el tipo de dato **Image sí debe ir con la I mayúscula** porque representa a la clase de C# que maneja imágenes.

¿El símbolo de interrogación (?) después de Image? sirve para activar la **nulabilidad estricta** de C#. Le indica explícitamente a C# que esa variable o parámetro **puede aceptar valores nulos (null)** sin que el compilador marque una advertencia o un error. En este caso es necesario porque si el PictureBox está vacío y no tiene foto, el método recibirá un null en lugar de una imagen real.

```
private byte[] ImageToByteArray(Image? image)
{
    if (image == null)
        return null;

    MemoryStream mMemoryStream = new MemoryStream();
    image.Save(mMemoryStream, ImageFormat.Png);
    return mMemoryStream.ToArray();
}
```

```
private byte[] ImageToByteArray(Image? image)
{
    if (image == null)
        return null;

    MemoryStream mMemoryStream = new
MemoryStream();
    image.Save(mMemoryStream, ImageFormat.Png);
    return mMemoryStream.ToArray();
}
return go(f, seed, [])
}
```